

Analiză Comparativă: Scala și Racket

Implementarea unui Calculator Funcțional

Jugaru Paul, Mincu Alin, Noje Vlad, Radoi Vlad

Tema Proiectului

Construirea unui **Calculator Funcțional** care poate să citească și să calculeze expresii matematice.

- Am ales două limbaje complet diferite pentru a vedea cum abordează fiecare aceeași problemă:
 - **Scala:** Un limbaj strict, care verifică totul înainte de rulare și se integrează cu Java.
 - **Racket:** Un limbaj flexibil, bazat pe liste, unde codul are aceeași formă cu datele.
- Obiectiv: Să comparăm cum definim, citim și calculăm structurile matematice (AST - Abstract Syntax Tree).

1. Scala: Ce este și cum funcționează?

- **Verificare Strictă:** Compilatorul găsește greșelile de tip înainte ca programul să ruleze.
- **Stil Hibrid:** Ne permite să folosim atât clase (ca în Java), cât și funcții matematice simple.
- **Interoperabilitate:** Putem folosi direct cod sau biblioteci scrise în Java.



Scala

Scala: Structura Calculatorului

```
1 sealed trait Expr
2 case class Num(v: Double) extends Expr
3 case class BinOp(op: String, l: Expr, r: Expr) extends Expr
```

Cum funcționează cuvintele cheie?

- **sealed trait**: Acționează ca o „umbrelă” care limitează tipurile de date. Programul știe exact că există doar `Num` și `BinOp`.
- **case class**: O clasă specială pentru stocarea datelor. O folosim ca să ne fie ușor să verificăm ulterior ce conține.
- **extends**: Arată că `Num` și `BinOp` fac parte din categoria generală `Expr`.

Scala: Parsare - Identificare Operator

```
1 case "(" :: op :: rest if Set("+", "-", "*", "/").contains(op) =>
2   for {
3     res1 <- parseTokens(rest)
4     // ... continuare procesare operanzi
```

Ce fac operatorii din cod?

- `::`: Taie o listă în bucăți. Extrage prima dată paranteza "(", apoi operatorul op, și păstrează restul.
- `if ... contains()`: Pune o condiție extra: procesează mai departe doar dacă op este un operator valid (+, -, *, /).
- `=>`: Desparte condiția de ceea ce vrem să se întâmple efectiv.

```
1 res1 <- parseTokens(rest)
2 (e1, rest1) = res1
3 res2 <- parseTokens(rest1)
4 (e2, rest2) = res2
5 // Right((BinOp(op, e1, e2), rest_final))
```

Cum extragem datele?

- for { }: Aici îl folosim pentru a lega o serie de operații care ar putea da eroare. Dacă un pas greșește, totul se oprește în siguranță.
- <-: Ia rezultatul dintr-o funcție și ni-l dă spre utilizare.
- (e1, rest1) =: Salvăm expresia găsită în e1 și păstrăm restul simbolurilor în rest1 pentru a căuta următorul element.

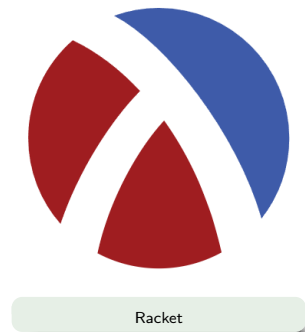
```
1 def eval(e: Expr): Double = e match {  
2   case Num(v) => v  
3   case BinOp("+", l, r) => eval(l) + eval(r)  
4 }
```

Ce face codul?

- `match`: Verifică tipul elementului. E un simplu număr sau e o operație cu două numere?
- `eval(l) + eval(r)`: Funcția se cheamă pe ea însăși ca să calculeze prima dată bucata din stânga (`l`), apoi pe cea din dreapta (`r`), și la final le adună.

2. Racket: Flexibilitate și Liste

- **Bazat pe Liste:** Datele și instrucțiunile împărtășesc aceeași structură uniformă. Totul este evaluat ca o listă între paranteze, ceea ce face logica programului foarte curată.
- **Extrem de Flexibil:** Nu declarăm tipurile în avans, lăsând programul să se adapteze natural în timpul execuției.
- **Ușor de Extins:** Racket ne permite nu doar să scriem funcții, ci chiar să extindem limbajul cu propriile noastre reguli gramaticale.



Racket: Tipuri de Date

```
1 (struct num (v) #:transparent)
2 (struct binop (op e1 e2) #:transparent)
3 (struct unop (op e) #:transparent)
```

Cum organizăm datele?

- În loc de clase complexe, folosim **structuri**, ca niste cutiuțe.
- num: Cutiuța pentru un număr simplu.
- binop / unop: Cutiuțe pentru operații
- #:transparent: Ne lasă să vedem ce e în cutiuță când o afișăm pe ecran, în loc să o țină ascunsă în memoria calculatorului.

Racket: Parsare - Numar Simplu

```
1 (define (parseaza expr)
2   (match expr
3     [(? number? n) (num n)]
4     ; ... restul regulilor
```

Cum citim datele?

- **match**: Este “inspectorul”. Ia expresia și o trece prin niște reguli de sus în jos, până găsește una care i se potrivește.
- **? number?**: **match** aplică un test și verifică dacă **expr** este doar un număr simplu.
- Dacă da, îi pune eticheta **n** și îl bagă în cutiuța lui: **(num n)**.

Racket: Parsare - Operații

```
1 [(list '+ e1 e2) (binop '+ (parseaza e1) (parseaza e2))]  
2 [(list 'sqrt e) (unop 'sqrt (parseaza e))]
```

- **list**: Verifică dacă expresia are structura corectă: un semn și două expresii (e1, e2) pentru operații binare, sau o singura expresie (e) pentru cele unare.
- **Apostroful (')**: Transformă textul în simbol ('+). Doar verifică dacă utilizatorul a scris semnul plus, NU face adunarea încă.
- **Recursivitatea**: Pentru că brațele arborelui (e1, e2 sau e) pot fi la rândul lor alte operații ascunse, le trimitem înapoi în funcția parseaza până devin numere simple.

Racket: Evaluare (Creierul Calculatorului)

```
1 (define (eval-expresie e)
2   (match e
3     [(num v) v]
4     [(binop '+ l r) (+ (eval-expresie l) (eval-expresie r))]))
```

- **Parcurgerea arborelui:** În această etapă, preluăm arborele abstract construit la parsare și îl calculăm recursiv.
- **Logica evaluării:** Dacă funcția întâlnește un număr (`num`), îi returnează direct valoarea. Dacă dă de o operație (`binop`), evaluează mai întâi brațele în adâncime, iar apoi o rezolvă.
- **Execuția reală:** (`+` ...) nu mai are apostrof! Acesta este momentul în care părăsim analiza de text și apelăm efectiv funcția matematică pentru a calcula rezultatul.

Racket: Vizualizare (Prefix $\rightarrow$ Infix)

```
1 (define (afiseaza-expresie ast)
2   (match ast
3     [(num n) (number->string n)]
4     [(binop op e1 e2)
5       (string-append "(" (afiseaza-expresie e1) " "
6                         (symbol->string op) " "
7                         (afiseaza-expresie e2) ")")])]
```

Conversia în format matematic standard:

- **Din AST în Infix:** Preluăm arborele abstract (care folosește formatul intern Prefix, ex: (+ 3 4)) și îl transformăm într-o expresie matematică normală, ușor de citit de către utilizator (format Infix, ex: (3 + 4)).
- **Concatenarea (string-append):** Combină parantezele, brațul din stânga evaluat recursiv, semnul operatorului și brațul din dreapta într-un singur șir de caractere (string) final.

```
1 (define (repl)
2   (display "> ")
3   (let ([input (read)])
4     (unless (eq? input 'exit)
5       (with-handlers ([exn:fail? (lambda (exn) (displayln ...))])
6         ; ... face parsarea, evaluarea si afisarea ...
7         (repl))))))
```

Buclo Infinită:

- **REPL**: Read, Eval, Print, Loop. Așteaptă comenzi, calculează, afișează, repetă.
- **with-handlers**: Dacă utilizatorul încearcă să împartă la zero, programul prinde eroarea în aer. Afișează mesajul, dar NU se blochează, ci așteaptă următoarea comandă!

3. Comparație și Performanță

Testul de Viteză

Scala rulează de **1.5 - 2 ori mai repede** decât Racket atunci când are de făcut calcule foarte lungi și repetate.

- **Găsirea erorilor:**

- În Scala, vezi imediat dacă ai greșit când scrii codul (eroare la compilare).
- În Racket, afli dacă ai greșit doar când pornești programul și ajunge la linia respectivă.

4. Lizibilitatea și Concluzii

- **Scala:** Codul este mai sigur și e mai ușor de înțeles de către alți programatori, ideal pentru proiecte mari.
- **Racket:** Codul e mult mai scurt, dar din cauza parantezelor multe, poate deveni o provocare să-l citești.
- **Concluzia noastră:** Scala este mai bună pentru zona de producție/industrie, iar Racket pentru testare rapidă și mediul academic.

 Wikipedia. *Scala (programming language)*.

https:

[//en.wikipedia.org/wiki/Scala_\(programming_language\)](https://en.wikipedia.org/wiki/Scala_(programming_language))

 Wikipedia. *Racket (programming language)*.

https:

[//en.wikipedia.org/wiki/Racket_\(programming_language\)](https://en.wikipedia.org/wiki/Racket_(programming_language))

 Google Gemini. *Model de limbaj AI utilizat pentru structurare.*

<https://gemini.google.com/>

Vă mulțumim!